
Modül 17 — GraphQL ve WebSocket

Ödevler

Spring Boot Edu



2026-09-24

İçindekiler

Nasıl Çalışılır	2
Ödev 1 — Yazarlar ve Kitapları (Kolay)	2
Ödev 2 — Sipariş Vermek (Orta)	3
Ödev 3 — Canlı Stok Bildirimleri (Orta)	3
Ek Meydan Okuma (İsteğe Bağlı)	4

Nasıl Çalışılır

1. Başlangıç kodu `modules/17-graphql-websocket/exercise/` altındadır. TODO yorumlarını bulun.
2. Ödevler Docker istemez: Katalog ve envanter bellektedir (catalog paketi, hazır verilmiştir).
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/17-graphql-websocket/exercise -am test
```

4. Tüm testler yeşil olduğunda ödev tamamdır.
5. Takılırsanız önce ipuçlarını, sonra `modules/17-graphql-websocket/solution/` altındaki çözümü okuyun.

Ödev 1 — Yazarlar ve Kitapları (Kolay)

Hedef: İstemciler kitaplarıyla birlikte bir yazar listesi istiyor. Şemada henüz bu alan yok ve kitaplar N+1 olmadan yüklenmeli.

Yapılacaklar:

- TODO 1a (`src/main/resources/graphql/schema.graphqls`) — Author tipine bir books alanı verin: Null olmayan Book'lardan oluşan, null olmayan bir liste.
- TODO 1b (`exercisel.AuthorController`) — `Query.authors`'ı (tüm yazarlar) ve `Query.author(id)`'yi (bilinmeyen bir id için null) gerçekleştirin.
- TODO 1c — `Author.books`'u gerçekleştirin. Bir sonucun tüm yazarlarının kitapları kataloğa **tek** bir çağrıyla yüklenmelidir.

İpuçları:

- Ders bölüm 3.1: `[Book!]!`.
- Ders bölüm 3.2: `@QueryMapping` ve `@Argument long id`. ID bir string olarak gelir ve Spring onu long'a çevirir.
- Ders bölüm 3.3: `catalog.findBooksOf(Collection<Long>)` ile `@BatchMapping Map<Author, List<Book>> books(List<Author> authors).Collectors.groupingBy(Book::authorId)` kitapları yazara göre gruplar.
- Kitabı olmayan bir yazar bile map'te olmalıdır (`List.of()`).

Kabul kriterleri: `Exercise1Test` içindeki 3 testin hepsi geçer (biri `catalog.bookQueries() == 1`'i kontrol eder).

Tahmini süre: 25 dakika

Ödev 2 — Sipariş Vermek (Orta)

Hedef: Bir `placeOrder` mutation'ı. Şema ve `OrderService` (doğrulama, envanter) hazır verilmiştir. Controller onları şemaya bağlar ve servisin exception'larını GraphQL hatalarına çevirir.

Yapılacaklar (`exercise2.OrderController`):

- TODO 2a — `Mutation.placeOrder`: Input'u `OrderService.place(...)`'e verin.
- TODO 2b — `Order.total`: `PlacedOrder`'da toplam yoktur. Bu yüzden birim fiyat × miktar toplamını hesaplayın.
- TODO 2c — `OrderLine.book`: `PlacedLine`'da yalnızca ISBN vardır. Katalogdaki kitabı döndürün.
- TODO 2d — hatalar: `InvalidOrderException` ve `OutOfStockException` → `BAD_REQUEST`, `UnknownBookException` → `NOT_FOUND`.

İpuçları:

- `@Argument OrderInput input` ile `@MutationMapping`: İç içe satır listesi `OrderLineInput` record'larına bağlanır.
- Burada Java tip adı şema tip adı değildir. Tipi Spring'e söyleyin: `@SchemaMapping(typeName = "Order")` ve `@SchemaMapping(typeName = "OrderLine")`.
- Bir `BigDecimal`, GraphQL `Float` olarak yazılır.
- Ders bölüm 3.4: İki exception için `@ExceptionHandler({InvalidOrderException.class, OutOfStockException.class})` ve `GraphQLErrorBuilder.newError(env).errorType(ErrorType.BAD_`

Kabul kriterleri: `Exercise2Test` içindeki 4 testin hepsi geçer.

Tahmini süre: 30 dakika

Ödev 3 — Canlı Stok Bildirimleri (Orta)

Hedef: Her sipariştten sonra mağazanın yönetim ekranı kitabın yeni stoğunu hemen gösterir. Yalnızca birkaç kopya kaldığında ayrı bir uyarı kanalı bilgilendirilir.

STOMP endpoint'i `/ws` ve `/topic` için broker hazır verilmiştir (`WebSocketConfiguration`). Envanter her değişiklikten sonra bir Spring application event'i yayınlar: `StockChanged(isbn, remaining)`.

Yapılacaklar (`exercise3.StockNotifier`):

- TODO 3a — Her `StockChanged` event'inde `/topic/stock/{isbn}`'e bir `StockLevel` gönderin (örneğin `/topic/stock/9780321336781`).
- TODO 3b — `LOW_STOCK`'tan (3) az kopya kaldığında `/topic/stock-alerts`'e ayrıca bir `StockAlert` gönderin.

İpuçları:

- StockChanged parametrelili bir @EventListener metodu event'leri alır.
- Ders bölüm 3.6: messaging.convertAndSend(destination, payload). Payload JSON'a çevrilir.
- Test siparişleri GraphQL ile verir (önce ödev 2 çözülmelidir) ve bir STOMP istemcisiyle dinler.

Kabul kriterleri: Exercise3Test içindeki iki test de geçer.

Tahmini süre: 20 dakika

Ek Meydan Okuma (İsteğe Bağlı)

Ödev 3'teki stok değişikliklerinin aynısını GraphQL üzerinden ileten bir stockChanged(isbn: ID!): Int! GraphQL subscription'ı ekleyin. Ders bölüm 3.5'teki gibi bir Sinks.Many kullanın ve executeSubscription() ile StepVerifier kullanan bir test yazın.